

Database Design

Rotana Spa Management System — PostgreSQL schema.

1. Technology

- **DBMS:** PostgreSQL 15+
- **Deployment:** Managed Neon (serverless) in this environment; compatible with any hosted PostgreSQL.
- **Driver:** pg (node-postgres) with connection pooling.
- **Schema:** public.
- **Conventions:**
 - id — integer / bigserial surrogate primary key.
 - company_id — tenant column on all tenant-owned tables; every query scopes by it.
 - Timestamps: created_at, updated_at (timestamptz where precision matters).
 - Soft delete where required: deleted_at (spa_management_records, spa_services).

2. ER overview

```
companies 1 — * users
companies 1 — * roles 1 — * role_permissions
companies 1 — * modules / company_modules (licensing)
companies 1 — * settings
companies 1 — * id_definitions 1 — * id_sequences

companies 1 — * membership_plans 1 — * membership_members 1 — * subscriptions
membership_members 1 — * rfid_cards / qr_passes / visit_sessions / gym_checkins
membership_members 1 — * spa_appointments / access_logs

companies 1 — * spa_facilities 1 — * rate_cards / day_tickets / spa_appointments
companies 1 — * entry_gates 1 — * access_logs

companies 1 — * spa_management_records      (generic operational records: services, therapists, trainers, classes, inventory, staff, towels)
companies 1 — * spa_visits 1 — * spa_visit_services
spa_visits 1 — 0..1 spa_service_orders
companies 1 — * spa_visit_counters          (per-company visit number sequence)

companies 1 — * audit_logs / notifications / staff / employees / appointments / customer_charges / membership_payments / demo_licenses
```

3. Table catalogue (36 tables)

3.1 Platform & tenancy

companies

Column	Type	Notes
id	integer PK	
name	varchar NOT NULL	
code	varchar	e.g. CMP-ROTANA

Column	Type	Notes
address / phone / email / website	text/varchar	
contact_person / contact_phone / contact_email	varchar	
tin	varchar	Tax ID
license_type	varchar	
status	varchar	active / ...
registration_date	date	
notes	text	
created_at / updated_at	timestamp	

modules and company_modules

- `modules` : code (e.g. `membership`), name, description, icon, `is_active`, `sort_order`.
- `company_modules` : `company_id` ↔ `module_id`, `is_enabled` → **license** of modules per company.

users

Column	Type	Notes
id	integer PK	
name	varchar	
email	varchar UNIQUE NOT NULL	login
password	text NOT NULL	bcrypt hash
role	varchar	<code>super_admin</code> / <code>admin</code> / <code>manager</code> / <code>receptionist</code> / <code>guest</code>
role_id	integer FK roles	optional granular role
is_active	boolean	
phone	varchar	
company_id	integer FK companies	null for super admin

roles and role_permissions

- `roles` : name, description, `company_id`, `is_active`.
- `role_permissions` : `role_id`, **resource**, `can_view`, `can_create`, `can_edit`, `can_delete`, `can_approve`. Unique (`role_id`, `resource`).

settings

- `company_id` (nullable = global), `key` (e.g. `company_name` , `company_address` , `company_phone` , `company_email` , `currency`), `value` , `description` .

id_definitions and id_sequences

- `id_definitions` : `company_id`, `entity_type`, `prefix`, `suffix`, `separator`, `pad_length`, `start_from`, `reset_type` (never/yearly/monthly/daily), `pattern`, `is_active`.
- `id_sequences` : `definition_id`, `company_id`, `period_key`, `current_value` — counter per period.

demo_licenses

- `license_key`, `company_id`, `company_name`, contact info, `issued_date`, `expiry_date`, `duration_days`, `status`, `issued_by`.

3.2 Membership

membership_plans

id, company_id, name, type (gym|spa|cafe|general), description, duration_days, price, currency, max_members, is_active, created_at, updated_at

membership_members

id, company_id, plan_id (FK), full_name, phone, email, id_number, address, photo_url, start_date (NOT NULL), end_date (NOT NULL), status (active|expired|...), customer_id (MEM-###), qr_code, notes, created_at, updated_at

subscriptions

id, company_id, member_id, plan_id, start_date, end_date (NOT NULL), billing_cycle, amount, status, auto_renew, payment_method, payment_reference, notes, freeze_start, freeze_end

membership_payments

id, company_id, member_id, amount, currency, payment_method, reference, payment_date, notes

rfid_cards

id, company_id, member_id, card_uid, type, status (active|inactive|lost|expired), issued_date, expiry_date

qr_passes

id, company_id, member_id, pass_type, qr_code, token, issued_date, expiry_date, max_uses, current_uses, status

day_tickets

id, company_id, ticket_number, guest_name, facility_id, rate_id, price, currency, qr_code, is_used, used_at, issued_by

3.3 Access & attendance

entry_gates

id, company_id, name, location, gate_type, reader_type, is_active

access_logs

id, company_id, gate_id, card_uid, member_id, access_type, method, status, created_at

visit_sessions

id, company_id, member_id, subscription_id, card_uid, facility_id, check_in_at, check_out_at, duration_minutes, source, notes

gym_checkins

id, company_id, member_id, card_uid, check_in_at, check_out_at, status, source

3.4 Spa facilities & pricing

spa_facilities

id, company_id, name, type (room|zone|pool|sauna|steam|cafe|gym|changing|other), capacity, description, is_active

rate_cards

id, company_id, name, facility_id (FK), service_type (membership|day_pass|session|facility|service), price, currency, duration_minutes, is_active

spa_services (legacy service catalogue)

id, company_id, name, category, duration, price, commission, room_required, description, is_active, is_deleted

appointments

id, company_id, customer_name, customer_phone, appointment_type, staff_id, service_id, start_time, end_time, status, notes

3.5 Generic operational records (workspaces)

spa_management_records — single store for all module workspaces

Column	Type	Notes
id	bigint PK	
company_id	integer NOT NULL	tenant
module_key	varchar NOT NULL	e.g. spa/services, spa/therapists, gym/trainers, gym/classes, inventory/products, staff/employees, operations/towel-management, ...
record_code	varchar NOT NULL	auto prefix + UUID8
title	varchar NOT NULL	primary display field
status	varchar NOT NULL	per-module status set
record_date	timestamptz	module date field
amount	numeric	module amount field
details	jsonb NOT NULL	module-specific fields
created_by / updated_by	integer	
created_at / updated_at / deleted_at	timestamptz	soft delete

The module definitions live in lib/spa-modules.ts ; each module declares its fields, statuses and derived-status rules (BMI, commission, stock, class full, towel status).

3.6 Visits & service orders (v34)

spa_visit_counters

company_id PK, current_value bigint, updated_at — per-company visit numbering.

spa_visits

Column	Type	Notes
id	bigint PK	
company_id	integer NOT NULL	
visit_no	varchar NOT NULL	SPA-000001 ; UNIQUE(company_id, visit_no)
member_id	integer FK membership_members	nullable (guest)
appointment_id	integer FK spa_appointments	UNIQUE per company when set

Column	Type	Notes
customer_name / customer_phone	varchar	
therapist_record_id	bigint FK spa_management_records	module <code>spa/therapists</code>
therapist_name	varchar	denormalised
status	varchar	<code>checked_in</code> / <code>assigned</code> / <code>in_treatment</code> / <code>finished</code> / <code>order_printed</code> / <code>handed_to_cashier</code> / <code>cancelled</code>
checked_in_at / treatment_started_at / finished_at	timestampz	
notes	text	
created_by / updated_by	integer	

Indexes: (company_id, status, checked_in_at DESC), (member_id), (therapist_record_id), partial unique (company_id, appointment_id) where appointment_id not null.

spa_visit_services

id, visit_id FK, company_id, service_record_id FK, service_code, service_name, quantity (≥1), notes, added_by — UNIQUE(visit_id, service_record_id).

spa_service_orders

Column	Type	Notes
id	bigint PK	
company_id	integer NOT NULL	
visit_id	bigint UNIQUE FK spa_visits	one order per visit
order_no	varchar NOT NULL	<code>SO-SPA-000001</code> ; UNIQUE(company_id, order_no)
status	varchar	<code>draft</code> / <code>printed</code> / <code>handed_to_cashier</code> / <code>void</code>
total_items	integer ≥ 0	
service_snapshot	jsonb NOT NULL	object; snapshot of service lines
generated_at / generated_by		
printed_at / printed_by / print_count		print tracking
handed_to_cashier_at		

3.7 Staff, inventory & misc

- `staff` — name, role, staff_type, commission_rate, specialization, license_number, email, phone, is_active, is_deleted.
- `employees` — company_id, full_name, phone, email, employee_type, user_id, is_active.
- `customer_charges` — company_id, customer_name, customer_phone, amount, status, is_paid, paid_at, external_receipt_no (POS receipt reference).
- `notifications` — company_id, user_id, title, message, type, is_read.
- `audit_logs` — company_id, user_id, action (CREATE/UPDATE/DELETE), table_name, record_id, old_values jsonb, new_values jsonb, ip_address, user_agent, created_at.

4. Check constraints (valid enums)

Table	Constraint	Allowed values
membership_plans	type	gym, spa, cafe, general
spa_facilities	type	room, zone, pool, sauna, steam, cafe, gym, changing, other
rate_cards	service_type	membership, day_pass, session, facility, service
spa_visits	status	checked_in, assigned, in_treatment, finished, order_printed, handed_to_cashier, cancelled
spa_service_orders	status	draft, printed, handed_to_cashier, void
spa_service_orders	service_snapshot	jsonb typeof object
spa_visit_services	quantity	> 0

5. Migration history

Migrations are sequential SQL files at the repository root:

File	Scope
db-migration.sql	Base schema: companies, users, roles, modules, membership, gates, sessions.
db-migration-v2.sql ... v32.sql	Incremental: payments, rfid/qr/day-tickets, spa_appointments, reports, audit, etc.
db-migration-v33.sql	spa_management_records + permissions (customers/gym/spa/inventory/staff/facilities/settings workspaces).
db-migration-v34.sql	spa_visit_counters , spa_visits , spa_visit_services , spa_service_orders + therapist/receptionist permissions (visit + draft service-order workflow).

Apply order: db-migration.sql → v2 ... v34.

6. Data-folder layout

```
database/  
├─ migrations/      # copy of db-migration*.sql (kept in repo root as the source of truth)  
├─ scripts/         # run-migration.cjs, run-migrations.js, seed-super-admin.js  
└─ seed-data/       # optional reference inserts (see scripts/_seed-dagi.cjs for a full demo dataset)
```

7. Seed data

- **Super admin:** super@rotanaspa.com / admin123 (company_id = NULL, platform role).
- **Demo tenant admin:** admin@rotanaspa.com / admin123 (company_id = 1).
- Full demo dataset (Dagi Spa model — Ethiopian spa & gym, ETB pricing) is generated by scripts/_seed-dagi.cjs and is **idempotent**.